

Raksha

R.A.K.S.H.A - Real-time AI Knowledge Security & Hazard Assessment

Operating Enterprise: YesPanchi Group of Companies

Document Title: Enterprise Technical Architecture, Port Specifications & Operations Manual

Document Code: RAKSHA-TECH-SPEC-2026-V2.4

Classification: Enterprise Confidential - Internal Engineering, DevOps & SecOps

Version Baseline: 2.4.0 (Enterprise Production Release)

Publication Date: October 2026

Executive Platform Overview

RAKSHA (Real-time AI Knowledge Security & Hazard Assessment) is an enterprise cybersecurity and AI governance platform developed by YesPanchi Group of Companies:

- **Centralized API Gateway:** High-performance proxy built on Go 1.26 **fasthttp** mediating enterprise microservices to downstream LLM providers (OpenAI, Anthropic, Bedrock, Ollama) with prompt guardrails, PII redaction, rate limiting, and virtual key budgeting.
- **Endpoint Browser Guard:** Workstation agent and local MitM proxy built on Python 3.11 **mitmproxy** that intercepts employee browser traffic to public Generative AI websites (ChatGPT, Claude, Gemini) to prevent data loss and unauthorized uploads.

Metadata Field	Platform Specification Details
Platform Name	Raksha (Real-time AI Knowledge Security & Hazard Assessment)
Enterprise Parent	YesPanchi Group of Companies
Dual-Plane Architecture	Go 1.26 Central API Gateway + Python 3.11 Endpoint MitM Interceptor
Primary Database	PostgreSQL 16 Enterprise (SCRAM-SHA-256 Authentication)
Deployment Engine	Multi-Container Docker Compose on Alpine Linux 3.23.4 Hardened Base
Supported Workstations	Microsoft Windows 10/11 (x64), Apple macOS (Apple Silicon / Intel), Linux

Table of Contents

Section No.	Chapter Title	Topic Scope & Subject Matter	Target Page
Chapter 1	Programming Languages & Version Matrix	Pinned versions for Go, React, TypeScript, Python, Alpine, and PostgreSQL	Page 3
Chapter 2	Network Architecture & Port Matrix	Complete port assignments (6000, 3100, 18182, 18103, 18195, 5432, 11434, 443)	Page 4
Chapter 3	Docker Installation & Command Handbook	1-line installation scripts, network setup, and daily container operations CLI	Page 5
Chapter 4	Cryptographic Secrets & Certificates Handbook	OpenSSL generation for .env secrets, Certbot HTTPS, and Root CA deployment	Page 6
Chapter 5	Production Deployment & Verification Matrix	4-step launch checklist, health check verification, and troubleshooting guide	Page 7

1. Programming Languages & Version Matrix

Every component of the Raksha platform is pinned to deterministic production versions to ensure reproducible builds, enterprise stability, and zero runtime drift:

Component / Layer	Technology	Exact Version	Source File in Repo	Technical Purpose & Function
Backend Core	Go (Golang)	1.26.4	go.work, core/go.mod	High-throughput gateway, proxying, and RBAC
HTTP Engine	valyala/fasthttp	v1.69.0	transports/raksha-http	Low-latency, memory-optimized HTTP engine
Database ORM	GORM	v1.31.1	framework/configstore	Schema migrations & PostgreSQL/SQLite driver
Embedded Database	mattn/go-sqlite3	v1.14.33	CGO Static Link	Fast local caching and offline fallback storage
Frontend Framework	React	19.2.3	ui/package.json	Modern component-based administration console
Frontend Language	TypeScript	5.9.2	ui/package.json	Strict type safety across all dashboard components
UI Build System	Vite	8.0.16	ui/package.json	High-speed frontend bundler and development server
Styling Engine	Tailwind CSS	4.1.12	ui/package.json	Responsive utility-first design system
Desktop Agent	Python	3.11+ / 3.12	apps/browser-guard	Workstation background agent & local MitM proxy
Interception Engine	mitmproxy	>= 10.0.0	apps/browser-guard	Real-time GenAI HTTP/HTTPS traffic analysis & DLP
Binary Compiler	PyInstaller	>= 6.0.0	apps/browser-guard	Compiles Python agent into standalone executable
Windows Installer	Inno Setup	6.2+	apps/browser-guard	Generates enterprise Windows silent setup installer
Container Base OS	Alpine Linux	3.23.4	Dockerfile.local	Minimal, security-hardened container runtime
Primary Database	PostgreSQL	16.x	docker-compose.yml	Production database for config, keys, and audit logs
Local AI Engine	Ollama	v0.3+ / v0.5+	docker-compose.yml	Offline local LLM model execution (Port 11434)

Key Architecture Principles: - Go fasthttp provides microsecond-level proxying latency for high-concurrency API inference. - Python mitmproxy delivers deep packet inspection and prompt redaction directly on employee endpoints. - Alpine Linux keeps the production container attack surface minimal (< 85 MB base).

2. Network Architecture & Port Matrix

Raksha requires specific network port assignments for inter-container communication, workstation proxying, and secure public ingress:

2.1 Complete Port Specification Table

Port Number	Protocol	Config Key	Scope / Binding	Service Name	Technical Function & Description
6000	TCP	APP_PORT	0.0.0.0:6000	raksha_tech	Primary Go backend. Serves API gateway, Web UI, and Browser Guard fleet reporting.
3100	TCP	UI_PORT	127.0.0.1:3100	Vite Dev Server	Local frontend development server (npm run dev in ui/). Inactive in production.
18182	TCP	PROXY_PORT	0.0.0.0:18182	Network Proxy	Optional shared MitM proxy container for centralized office PAC deployments.
18103	TCP	RAKSHA_PROXY_ADD_R	127.0.0.1:18103	Laptop Guard Proxy	Local MitM proxy running on each employee laptop to inspect outbound GenAI traffic.
18195	TCP	PAC_HTTP_PORT	127.0.0.1:18195	Laptop PAC Server	Local HTTP server serving the dynamic Proxy Auto-Configuration (PAC) script to the OS.
5432	TCP	DB_PORT	Internal VPC	PostgreSQL	Production relational database storing configuration, tenants, keys, and audit logs.
11434	TCP	OLLAMA_URL	Docker Bridge	Ollama AI Engine	Local AI inference engine serving offline open-source models via 1panel-network.
80	TCP	System Port	0.0.0.0:80	Nginx / Caddy	Inbound public HTTP port for Certbot ACME domain validation and HTTPS redirect.
443	TCP	System Port	0.0.0.0:443	Nginx / Caddy	Public secure HTTPS edge entry point terminating SSL/TLS certificates.

2.2 Perimeter Firewall Security Rules

+-----+				
FIREWALL ACCESS CONTROL MATRIX				
+-----+-----+-----+-----+-----+				
Traffic Policy Port Allowed Source IP Security Policy & Purpose				
+-----+-----+-----+-----+-----+				
PUBLIC INGRESS 443 0.0.0.0/0 (Internet) Main entry point terminating TLS for UI/API PUBLIC INGRESS				
80 0.0.0.0/0 (Internet) ACME HTTP-01 challenge & redirect to HTTPS INTERNAL PRIVATE 6000				
127.0.0.1 / Nginx Host Protect Go backend from direct public access INTERNAL PRIVATE 5432 App				
Containers / VPC Restrict PostgreSQL database to VPC only INTERNAL PRIVATE 11434 App Containers /				
VPC Restrict Ollama LLM to internal backend ENDPOINT LOOPBACK 18103 127.0.0.1 Only Workstation				
proxy bound strictly to local ENDPOINT LOOPBACK 18195 127.0.0.1 Only Workstation PAC server bound				
to local				
+-----+-----+-----+-----+-----+				

3. Docker Installation & Command Handbook

3.1 Download & Install Docker (One-Liner Commands)

- **Ubuntu / Debian Linux (20.04, 22.04, 24.04 LTS):**

```
curl -fsSL https://get.docker.com | sh && sudo systemctl enable --now docker && sudo usermod -aG docker $USER
```

- **CentOS / RHEL 8 & 9 / Rocky Linux:**

```
sudo dnf install -y dnf-plugins-core && sudo dnf config-manager --add-repo  
https://download.docker.com/linux/centos/docker-ce.repo && sudo dnf install -y docker-ce  
docker-compose-plugin && sudo systemctl enable --now docker
```

- **Microsoft Windows 10 / 11:**

1. Open PowerShell as Administrator and run: **wsl --install**
2. Download Docker Desktop from: <https://docs.docker.com/desktop/install/windows-install/>

3.2 Docker Network Prerequisites (Run Once)

```
docker network create 1panel-network docker network create raksha-network
```

3.3 Daily Docker Operations Command Handbook

```
# --- STARTUP & BUILD OPERATIONS --- # Start Raksha backend in background (detached mode) docker compose up  
-d # Start backend AND optional shared network proxy docker compose --profile network-proxy up -d # Rebuild  
and launch containers after any code update docker compose up -d --build # Full clean rebuild without Docker  
cache docker compose build --no-cache && docker compose up -d # --- MONITORING & LOG OPERATIONS --- # Check  
status and health of running containers docker compose ps # Follow real-time live logs of the backend  
container docker compose logs -f --tail=100 raksha_tech # Check container CPU %, RAM consumption, and I/O  
docker stats raksha_tech # --- IN-CONTAINER DEBUGGING OPERATIONS --- # Open an interactive shell inside the  
Raksha container docker exec -it raksha_tech /bin/sh # Test backend healthcheck from inside container docker  
exec -it raksha_tech wget -qO- http://127.0.0.1:6000/health # --- STOP, RESTART & MAINTENANCE --- # Restart  
backend gracefully (e.g. after editing .env) docker compose restart raksha_tech # Stop and remove containers,  
networks, and internal links docker compose down -v # Clean up stopped containers, unused networks, and  
dangling images docker system prune -f
```

4. Cryptographic Secrets & Certificates Handbook

All cryptographic keys, authentication tokens, and SSL certificates must be generated using cryptographically secure random sources prior to production launch:

4.1 Secrets for .env File

Environment Secret Key	Standard	Technical Purpose	Exact Generation Command
RAKSHA_GUARD_SECRET	256-bit Random Base64	Fleet Desktop Agent Mutual Auth Token	openssl rand -base64 32
RAKSHA_ENCRYPTION_KEY	256-bit Random Base64	AES-256 DB Credential Encryption at Rest	openssl rand -base64 32
PASSWORD_RESET_SECRET	384-bit Random Base64	User Password Recovery HMAC Token	openssl rand -base64 48
RAKSHA_METRICS_TOKEN	256-bit Random Hex	Prometheus Bearer Token for /metrics	openssl rand -hex 32
CLUSTER_REPLICATE_SECRET	384-bit Random Base64	Multi-Node Cluster State Replication	openssl rand -base64 48

```
# Automated one-liner to generate and inject all secrets directly into .env: sed -i
"s|^RAKSHA_GUARD_SECRET=.*|RAKSHA_GUARD_SECRET=$(openssl rand -base64 32)|" .env sed -i
"s|^RAKSHA_ENCRYPTION_KEY=.*|RAKSHA_ENCRYPTION_KEY=$(openssl rand -base64 32)|" .env sed -i
"s|^PASSWORD_RESET_SECRET=.*|PASSWORD_RESET_SECRET=$(openssl rand -base64 48)|" .env sed -i
"s|^RAKSHA_METRICS_TOKEN=.*|RAKSHA_METRICS_TOKEN=$(openssl rand -hex 32)|" .env sed -i
"s|^CLUSTER_REPLICATE_SECRET=.*|CLUSTER_REPLICATE_SECRET=$(openssl rand -base64 48)|" .env
```

4.2 Public & Private SSL/TLS Certificate Generation

- **Public Let's Encrypt HTTPS (Certbot):**

```
sudo certbot certonly --standalone -d raksha.yourcompany.com --email secops@yourcompany.com --agree-tos
```

- **Private Self-Signed TLS Certificate (OpenSSL 4096-bit RSA with SAN):**

```
openssl req -x509 -nodes -days 1095 -newkey rsa:4096 \ -keyout raksha_server.key -out raksha_server.crt \
-subj "/C=US/ST=State/L=City/O=YesPanchi/OU=IT/CN=raksha.yourcompany.com" \ -addext
"subjectAltName=DNS:raksha.yourcompany.com,DNS:localhost,IP:127.0.0.1"
```

4.3 Browser Guard Root CA Generation & Installation

```
# 1. Generate 4096-bit Root CA Key and 10-year Certificate for MitM proxy openssl genrsa -out raksha-ca.key
4096 openssl req -x509 -new -nodes -key raksha-ca.key -sha256 -days 3650 \ -out raksha-ca.crt -subj
"/CN=Raksha Root CA - Enterprise AI Guard" cat raksha-ca.key raksha-ca.crt > mitmproxy-ca.pem # 2. Deploy
Root CA to Workstation Trust Stores: # Windows (CMD Admin) : certutil -addstore -f "Root" raksha-ca.crt #
macOS (Terminal) : sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain
raksha-ca.crt # Linux (Workstation) : sudo cp raksha-ca.crt /usr/local/share/ca-certificates/ && sudo
update-ca-certificates
```

5. Production Deployment & Verification Matrix

5.1 Quick-Start 4-Step Deployment Checklist

+-----+			
PRODUCTION DEPLOYMENT CHECKLIST			
+-----+			
Step	Action Task	Terminal Command to Execute	
+-----+ 1			
Copy Environment File	cp .env.example .env	2 Generate Secrets	Populate RAKSHA_GUARD_SECRET & RAKSHA_ENCRYPTION_KEY in .env
+-----+ 3			
Create Docker Network	docker network create 1panel-network	4	
+-----+			
Compile & Start App	docker compose up -d --build		
+-----+			

5.2 Health Checks & Operational Verification

1. Verify Go Backend Healthcheck Endpoint curl -i http://localhost:6000/health # Expected: HTTP/1.1 200 OK {"status":"healthy"} # 2. Verify Container Process Status docker compose ps # Expected: raksha_tech (healthy) Up X minutes 0.0.0.0:6000->6000/tcp # 3. Test PostgreSQL Database Connectivity from Host nc -zv 127.0.0.1 5432 # Expected: Connection to 127.0.0.1 port 5432 [tcp/postgresql] succeeded! # 4. Test Local Ollama Model Server Connectivity curl -i http://127.0.0.1:11434/api/tags # Expected: HTTP/1.1 200 OK with JSON array of installed models
--

5.3 Operational Diagnostics & Resolution Matrix

Symptom / Observed Error	Root Cause Analysis	Corrective CLI Resolution
failed to connect to PostgreSQL on port 5432	Database container down or firewall blocking port	Run docker compose ps to inspect DB; check DB credentials in .env ; verify host port.
network 1panel-network not found	External network was not created prior to launch	Run docker network create 1panel-network , then rerun docker compose up -d .
Browser Guard: 401 Unauthorized on /api/*	Mismatched RAKSHA_GUARD_SECRET between client & backend	Run python apps/browser-guard/scripts/sync_config_from_env.py to synchronize keys.
Browser displays ERR_CERT_AUTHORITY_INVALID	Root CA certificate not installed in client trust store	Install raksha-ca.crt into client OS store (certutil on Windows, security on macOS).
Port 6000 already in use / bind: address already in use	Another process is occupying host port 6000	Run `netstat -ano`
UI displays blank screen or 404 on assets	Web UI static bundle missing from Go binary	Run npm run build:raksha in ui/ , then rebuild with docker compose up -d --build .

Platform Support & Enterprise Maintenance

- **Lead Repository:** **d:\unifai_project**
- **Core Configuration:** **.env** and **configs/config.json**
- **Engineering Parent:** Raksha SecOps & Cloud Infrastructure Group, YesPanchi Group of Companies